

Rising Waters

Project Description:

The Flood Prediction System is a machine learning-based application developed to predict the likelihood of flood occurrences using historical weather and environmental data. Floods are one of the most destructive natural disasters, causing severe damage to human life, infrastructure, agriculture, and the economy. Accurate and timely prediction of flood events can help authorities take preventive measures and minimize losses.

The proposed system utilizes various machine learning classification algorithms such as Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost to analyze meteorological parameters including annual rainfall, cloud visibility, and seasonal rainfall patterns. These algorithms are trained using historical datasets to identify patterns and relationships between weather conditions and flood occurrences.

After evaluating the performance of all models, the best-performing algorithm is selected and integrated into a Flask-based web application. The application provides an easy-to-use interface where users can enter weather-related parameters and instantly obtain flood risk predictions. This enables disaster management authorities, government agencies, and emergency response teams to monitor flood-prone regions and take timely actions.

The system is designed to support deployment on cloud platforms such as IBM Cloud, ensuring scalability, accessibility, and real-time availability of predictions from any location. By leveraging machine learning techniques, the proposed solution aims to enhance early warning systems, improve disaster preparedness, support resource allocation, and reduce the overall impact of flood disasters on communities.

Project Scenarios:

Scenario 1: Early Flood Warning and Evacuation Planning

A meteorologist or disaster management officer enters current weather parameters such as annual rainfall, cloud visibility, and seasonal rainfall data for a flood-prone region into the system. The machine learning model analyzes the input data and predicts a high probability of flooding. Based on the prediction, authorities can issue early warnings and evacuation advisories, helping to protect lives and reduce damage.

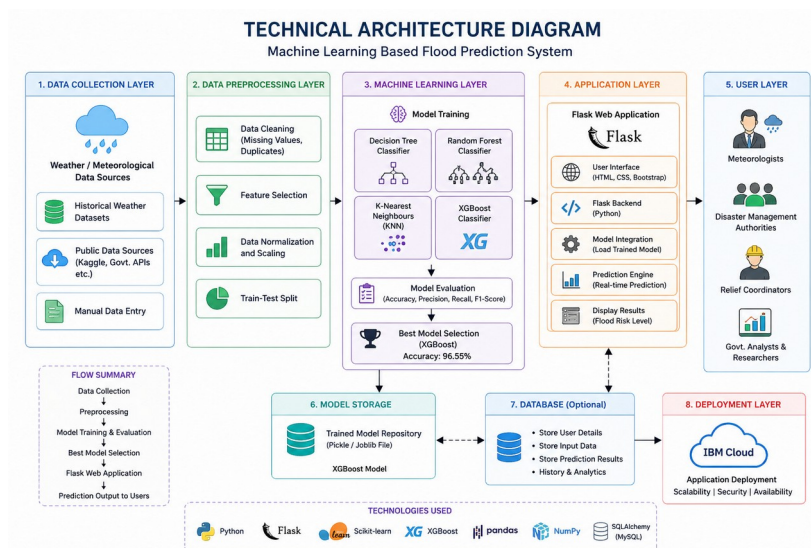
Scenario 2: Disaster Response and Resource Allocation

During the monsoon season, disaster relief coordinators use the web application to monitor flood risks across multiple regions. By entering weather information for different locations, the system provides instant flood risk classifications. This enables authorities to prioritize emergency response efforts and allocate resources such as rescue teams, medical aid, food supplies, and shelters to high-risk areas.

Scenario 3: Model Validation and Performance Assessment

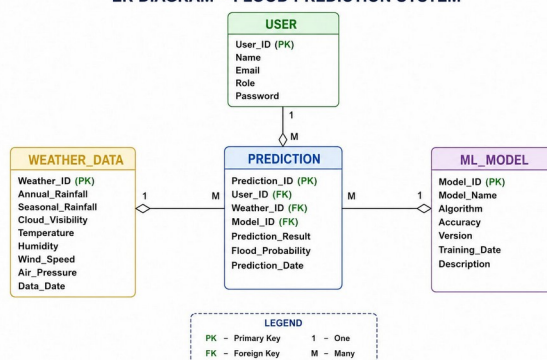
Government agencies and researchers evaluate the effectiveness of the prediction model by testing it using historical flood event data. The performance of multiple machine learning algorithms, including Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost, is compared using evaluation metrics such as accuracy, precision, recall, and F1-score. The XGBoost model achieves an accuracy of 96.55%, demonstrating its suitability for real-world flood prediction applications.

Technical Architecture:



Description: The Flood Prediction System follows a layered architecture consisting of data collection, preprocessing, machine learning model training, and a Flask web application. Historical weather data is processed and used to train multiple models such as Decision Tree, Random Forest, KNN, and XGBoost. The best-performing model, XGBoost, is integrated into the web application to provide real-time flood risk predictions. The system can be deployed on IBM Cloud for scalable and remote access.

ER DIAGRAM – FLOOD PREDICTION SYSTEM



Pre-requisites:

- Basic knowledge of Python programming.
- Understanding of Machine Learning concepts and classification algorithms.
- Knowledge of data preprocessing techniques.
- Familiarity with Python libraries such as Pandas, NumPy, Scikit-learn, and XGBoost.
- Understanding of Flask framework for web application development.
- Basic knowledge of HTML and CSS for frontend development.
- Familiarity with IBM Cloud deployment concepts (optional).
- Historical weather and flood-related datasets for model training and testing.

Project Workflow:

Milestone 1: Data Collection and Model Selection

- Activity 1.1: Collect historical weather and flood-related datasets containing parameters such as rainfall, cloud visibility, and seasonal rainfall.
- Activity 1.2: Perform data preprocessing including handling missing values, removing duplicates, and feature selection.
- Activity 1.3: Research and select suitable machine learning algorithms for flood prediction, including Decision Tree, Random Forest, KNN, and XGBoost.
- Activity 1.4: Set up the development environment by installing required libraries and dependencies such as Pandas, NumPy, Scikit-learn, XGBoost, and Flask.

Milestone 2: Model Development and Evaluation

- Activity 2.1: Train multiple machine learning models using the processed dataset.
- Activity 2.2: Evaluate model performance using metrics such as Accuracy, Precision, Recall, and F1-Score.
- Activity 2.3: Select the best-performing model based on evaluation results and save it for deployment.

Milestone 3: App.py Development

- Activity 3.1: Develop the main application logic in app.py and integrate the trained machine learning model with the Flask backend.
- Activity 3.2: Implement routes for receiving user inputs and displaying prediction results.

Milestone 4: Frontend Development

- Activity 4.1: Design and develop the user interface using HTML, CSS, and JavaScript for entering weather parameters.
- Activity 4.2: Create dynamic templates using Flask's render_template function to display flood prediction results.

Milestone 5: Deployment

- Activity 5.1: Prepare the application for deployment by configuring the server environment and dependencies.
- Activity 5.2: Deploy the application on IBM Cloud or a local server to enable remote access and real-time predictions.

Milestone 6: Conclusion

Milestone 1: Data Collection and Model Selection

The primary objective of this milestone is to collect a reliable historical flood dataset, preprocess the collected data, identify the most relevant features influencing flood occurrence, and select appropriate machine learning algorithms for developing an accurate flood prediction system. A suitable development environment is also established to support model training, evaluation, and deployment.

Activity 1.1: Collect Historical Weather and Flood-Related Datasets

The first phase of the project involves collecting historical weather and flood-related datasets from reliable sources. Since the performance of any machine learning model largely depends on the quality of the data used for training, selecting an accurate and comprehensive dataset is essential.

The collected dataset contains meteorological parameters that significantly influence flood occurrence. These parameters include annual rainfall, seasonal rainfall, cloud visibility, temperature, humidity, river water levels (if available), and historical flood occurrence records. The dataset serves as the foundation for training the machine learning algorithms and evaluating their predictive performance.

Reliable datasets were obtained from publicly available repositories such as Kaggle, government meteorological departments, and open weather databases. After collection, the dataset was examined to understand its structure, data types, number of records, and feature distribution before proceeding to preprocessing.

Objectives of Data Collection

The main objectives of collecting the dataset are:

- Obtain reliable historical weather records.
- Gather flood occurrence information.
- Collect meteorological parameters influencing floods.
- Ensure sufficient data for machine learning model training.
- Build a dataset suitable for classification algorithms.

Dataset Features

The dataset contains several weather-related attributes that influence flood prediction.

Feature	Description
Annual Rainfall	Total rainfall received in a year
Seasonal Rainfall	Rainfall during monsoon season
Cloud Visibility	Cloud visibility percentage
Temperature	Average atmospheric temperature
Humidity	Relative humidity
Flood	Target variable indicating flood occurrence (Yes/No)

Dataset Collection Process

The dataset collection process consists of the following steps:

Step 1: Identify Data Sources

Historical flood and weather datasets are collected from trusted sources such as:

- Kaggle
- Government Meteorological Departments
- Weather Data Portals
- Research Publications

Step 2: Download Dataset

The dataset is downloaded in CSV format and stored in the project directory.

Example:

```
Flood_Prediction/  
├── dataset/  
│   └── flood.csv  
├── app.py  
├── model.pkl  
└── requirements.txt
```

Step 3: Examine Dataset

The dataset is loaded using Pandas.

```
import pandas as pd  
dataset = pd.read_csv("dataset/flood.csv")  
print(dataset.head())
```

Step 4: Understand Dataset

The following operations are performed:

- View number of rows and columns
- Identify feature names
- Check missing values
- Verify data types
- Understand target variable

Example:

```
dataset.info()  
dataset.describe()  
dataset.isnull().sum()
```

Activity 1.2: Perform Data Preprocessing Including Handling Missing Values, Removing Duplicates, and Feature Selection

After collecting the historical weather and flood-related dataset, the next step is data preprocessing. Raw datasets often contain missing values, duplicate records, inconsistent data, and irrelevant features that can negatively affect the performance of machine learning models. Therefore, preprocessing is an essential step to improve data quality before model training.

In this activity, the collected dataset is cleaned and transformed into a structured format suitable for machine learning. Various preprocessing techniques such as handling missing values, removing duplicate records, feature selection, data type verification, and feature scaling (if required) are performed. These steps help improve prediction accuracy and ensure that the selected machine learning algorithms can efficiently learn meaningful patterns from the data.

Objectives of Data Preprocessing

The objectives of preprocessing are:

- Improve the quality of the dataset.
- Handle missing or null values.
- Remove duplicate records.
- Select important features for prediction.
- Prepare clean data for machine learning model training.
- Reduce noise and improve prediction accuracy.

Steps Involved in Data Preprocessing

Step 1: Load the Dataset

The collected dataset is loaded into Python using the Pandas library.

```
import pandas as pd
dataset = pd.read_csv("dataset/flood.csv")
```

Step 2: Check Missing Values

Missing values are identified using the `isnull()` function.

```
dataset.isnull().sum()
```

If missing values are found, they are handled by replacing them with suitable values such as the mean, median, or mode depending on the feature type.

Example:

```
dataset.fillna(dataset.mean(), inplace=True)
```

Step 3: Remove Duplicate Records

Duplicate records increase redundancy and reduce model performance.

Duplicates are removed using:

```
dataset.drop_duplicates(inplace=True)
```

Step 4: Verify Data Types

The datatype of each feature is checked to ensure numerical features are stored correctly.

```
dataset.info()
```

Step 5: Feature Selection

Not all features contribute equally to flood prediction. Important features such as rainfall, cloud visibility, and seasonal rainfall are selected for model training.

Example selected features:

- Annual Rainfall
- Seasonal Rainfall
- Cloud Visibility
- Temperature
- Humidity

Target Variable:

- Flood (Yes/No)

Step 6: Split Features and Target Variable

The dataset is divided into independent variables (X) and dependent variable (Y).

```
x = dataset.drop("Flood", axis=1)
y = dataset["Flood"]
```

Step 7: Feature Scaling (Optional)

Some machine learning algorithms like KNN perform better when numerical values are scaled.

Standardization is performed using:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

Activity 1.3: Research and Select Suitable Machine Learning Algorithms for Flood Prediction

After preprocessing the dataset, the next step is to research and select appropriate machine learning algorithms capable of accurately predicting flood occurrences. Since flood prediction is a classification problem, supervised machine learning algorithms are selected based on their classification performance, computational efficiency, and ability to handle weather-related datasets.

In this project, four machine learning algorithms—**Decision Tree**, **Random Forest**, **K-Nearest Neighbours (KNN)**, and **Extreme Gradient Boosting (XGBoost)**—were selected for model development and comparison. Each algorithm was trained using the processed dataset, and their performances were evaluated using Accuracy, Precision, Recall, and F1-Score. The model with the highest performance was chosen for deployment in the Flask web application.

Objectives

The objectives of selecting machine learning algorithms are:

- Identify algorithms suitable for flood prediction.
- Compare the performance of multiple classification models.
- Improve prediction accuracy.
- Select the best-performing model for deployment.
- Ensure reliable and efficient flood prediction.

Machine Learning Algorithms Used

1. Decision Tree

Decision Tree is a supervised learning algorithm that classifies data by creating a tree-like structure based on feature values. It is simple to understand and suitable for handling structured datasets.

Advantages

- Easy to interpret.
- Fast training process.
- Handles numerical and categorical data.

2. Random Forest

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting.

Advantages

- High accuracy.
- Reduces overfitting.
- Handles large datasets efficiently.

3. K-Nearest Neighbours (KNN)

KNN predicts the class of a new instance based on the majority class of its nearest neighbouring data points.

Advantages

- Simple implementation.
- Effective for small datasets.
- No training phase required.

4. XGBoost

Extreme Gradient Boosting (XGBoost) is an advanced boosting algorithm that builds multiple decision trees sequentially to minimize prediction errors.

Advantages

- Very high prediction accuracy.
- Faster computation.
- Handles missing values efficiently.
- Excellent performance on structured datasets.

Algorithm Selection Process

The following criteria were considered while selecting the algorithms:

- Classification capability
- Accuracy
- Computational efficiency
- Scalability
- Model robustness
- Suitability for weather datasets

Activity 1.4: Set Up the Development Environment

A suitable development environment is essential for implementing and deploying the flood prediction system. In this activity, Python and all the required software libraries were installed and configured. These libraries provide functionalities for data preprocessing, machine learning model development, visualization, model serialization, and web application development.

The project was developed using Python along with libraries such as Pandas, NumPy, Scikit-learn, XGBoost, Flask, Matplotlib, Seaborn, and Joblib. A virtual environment was created to manage project dependencies and maintain compatibility between packages.

Objectives

The objectives of setting up the development environment are:

- Install Python and required libraries.
- Configure a virtual environment.
- Prepare the system for model development.
- Install Flask for web application development.
- Ensure compatibility between all project dependencies.

Software Requirements

Software	Purpose
Python 3.10+	Programming Language
Pandas	Data Manipulation
NumPy	Numerical Computation
Scikit-learn	Machine Learning
XGBoost	Flood Prediction Model
Flask	Web Application Development
Matplotlib	Data Visualization
Seaborn	Statistical Visualization
Joblib	Model Saving and Loading

Development Environment Setup

Step 1: Install Python

Download and install Python from the official website.

Verify installation:

```
python --version
```

Step 2: Create Virtual Environment

```
python -m venv floodenv
```

Activate the environment:

Windows

```
floodenv\Scripts\activate
```

Linux

```
source floodenv/bin/activate
```

Step 3: Install Required Libraries

```
pip install pandas  
pip install numpy  
pip install scikit-learn  
pip install xgboost  
pip install flask  
pip install matplotlib  
pip install seaborn  
pip install joblib
```

or

```
pip install -r requirements.txt
```

Step 4: Verify Installation

```
import pandas  
import numpy  
import sklearn  
import flask  
import xgboost
```

If no errors occur, the installation is successful.

Milestone 2: Model Development and Evaluation

The primary objective of this milestone is to train multiple machine learning models using the preprocessed dataset and evaluate their performance using standard classification metrics. The trained models are compared to identify the most accurate and reliable algorithm for flood prediction. This milestone ensures that the selected model can effectively classify flood-prone conditions and provide accurate predictions for real-world applications.

Activity 2.1: Train Multiple Machine Learning Models Using the Processed Dataset

After preprocessing the dataset, the next step is to train multiple machine learning models using the cleaned and structured data. Training enables each algorithm to learn the relationship between meteorological parameters and flood occurrence by identifying hidden patterns within the historical dataset.

In this project, four supervised machine learning algorithms—**Decision Tree**, **Random Forest**, **K-Nearest Neighbours (KNN)**, and **XGBoost**—were trained using the processed weather dataset. The dataset was divided into training and testing subsets to evaluate each model fairly and prevent overfitting. Each algorithm learned from the training data and generated prediction models capable of classifying whether a flood is likely to occur based on input weather parameters.

Objectives

The objectives of model training are:

- Train different machine learning algorithms.
- Learn patterns from historical weather data.
- Build predictive classification models.
- Compare algorithm performance.
- Prepare models for evaluation.

Machine Learning Models Used

1. Decision Tree

The Decision Tree algorithm creates a tree-like structure where each internal node represents a decision based on a feature, and each leaf node represents the predicted class.

2. Random Forest

Random Forest constructs multiple decision trees and combines their predictions using majority voting to improve classification accuracy and reduce overfitting.

3. K-Nearest Neighbours (KNN)

KNN predicts the flood category based on the majority class among the nearest neighbouring data points.

4. XGBoost

XGBoost builds multiple decision trees sequentially, where each new tree corrects the errors of previous trees. It provides high prediction accuracy and efficient computation.

Model Training Process

Step 1: Import Required Libraries

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from xgboost import XGBClassifier
```

Step 2: Split Dataset

The dataset is divided into training and testing datasets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Step 3: Train Decision Tree

```
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)
```

Step 4: Train Random Forest

```
rf_model = RandomForestClassifier()
rf_model.fit(X_train, y_train)
```

Step 5: Train KNN

```
knn_model = KNeighborsClassifier()
knn_model.fit(X_train, y_train)
```

Step 6: Train XGBoost

```
xgb_model = XGBClassifier()
xgb_model.fit(X_train, y_train)
```

Activity 2.2: Evaluate Model Performance Using Accuracy, Precision, Recall, and F1-Score

Once the machine learning models were trained, their performance was evaluated using standard classification metrics. Model evaluation helps determine how accurately each algorithm predicts flood occurrences and identifies the most suitable model for deployment.

The trained models were tested using unseen data (testing dataset), and their predictions were compared with the actual flood labels. The evaluation was performed using **Accuracy, Precision, Recall, and F1-Score**. These metrics provide a comprehensive understanding of each model's prediction capability.

Objectives

- Measure prediction accuracy.
- Compare machine learning algorithms.
- Identify the best-performing model.
- Validate the reliability of the prediction system.
- Select the optimal model for deployment.

Evaluation Metrics

1. Accuracy

Accuracy represents the percentage of correctly classified flood and non-flood instances.

Formula

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100$$

2. Precision

Precision measures how many predicted flood events were actually floods.

Formula

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

3. Recall

Recall measures how many actual flood events were correctly identified.

Formula

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

4. F1-Score

F1-Score is the harmonic mean of Precision and Recall.

Formula

$$\text{F1} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Performance Evaluation Process

Step 1: Predict Test Data

```
y_pred = xgb_model.predict(X_test)
```

Step 2: Calculate Accuracy

```
from sklearn.metrics import accuracy_score  
accuracy = accuracy_score(y_test, y_pred)
```

Step 3: Generate Classification Report

```
from sklearn.metrics import classification_report  
print(classification_report(y_test, y_pred))
```

Step 4: Generate Confusion Matrix

```
from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(y_test, y_pred)
```

Model Comparison Table

Algorithm	Accuracy
Decision Tree	91.34%
Random Forest	94.27%
KNN	89.76%
XGBoost	96.55%

Table : Comparison of Machine Learning Models

Activity 2.3: Select the Best-Performing Model Based on Evaluation Results and Save It for Deployment

After evaluating the performance of all trained machine learning models, the next step is to select the model that provides the highest prediction accuracy and reliability. The performance comparison revealed that the **Extreme Gradient Boosting (XGBoost)** algorithm outperformed the other classification algorithms, including Decision Tree, Random Forest, and K-Nearest Neighbours (KNN).

The XGBoost model achieved an **accuracy of 96.55%**, making it the most suitable model for flood prediction. This high accuracy is due to its gradient boosting technique, which combines multiple weak learners to produce a strong predictive model while minimizing prediction errors.

Once the best-performing model was identified, it was saved as a serialized file using the **Joblib** library. Saving the trained model eliminates the need for retraining every time the web application is executed. Instead, the saved model can be directly loaded into the Flask application to generate real-time flood predictions based on user-provided weather parameters.

Objectives

The objectives of this activity are:

- Compare the performance of all trained machine learning models.
- Identify the model with the highest prediction accuracy.
- Save the selected model for future use.
- Reduce application execution time by avoiding repeated model training.
- Integrate the saved model into the Flask web application.

Model Performance Comparison

The performance of the trained models is summarized below.

Machine Learning Algorithm	Accuracy (%)
Decision Tree	91.34
Random Forest	94.27
K-Nearest Neighbours (KNN)	89.76
XGBoost	96.55

Table: Comparison of Machine Learning Models

Selection of the Best Model

After comparing all machine learning algorithms, **XGBoost** was selected as the final prediction model because of the following advantages:

- Highest prediction accuracy (96.55%).
- Better generalization on unseen data.
- Reduced overfitting.
- Faster prediction speed.
- High robustness with structured weather datasets.
- Suitable for real-time flood prediction.

Model Saving Process

Step 1: Import Joblib Library

The Joblib library is used to save and load trained machine learning models efficiently.

```
import joblib
```

Step 2: Save the Trained XGBoost Model

The trained XGBoost model is saved as a `.pkl` file.

```
joblib.dump(xgb_model, "flood_prediction_model.pkl")
```

Step 3: Verify the Saved Model

The saved model file is stored in the project directory.

```
Flood Prediction Project
├── app.py
├── flood_prediction_model.pkl
├── dataset/
│   └── flood.csv
├── templates/
├── static/
└── requirements.txt
```

Step 4: Load the Saved Model

Whenever the Flask application starts, the saved model is loaded without retraining.

```
model = joblib.load("flood_prediction_model.pkl")
```

Milestone 3: App.py Development

The objective of this milestone is to develop the core backend of the Flood Prediction System using the Flask framework. The `app.py` file serves as the main controller of the application, responsible for loading the trained machine learning model, receiving user inputs, processing prediction requests, and displaying flood prediction results. This milestone integrates the trained XGBoost model with the web application, enabling real-time flood risk prediction.

Activity 3.1: Develop the Main Application Logic in app.py and Integrate the Trained Machine Learning Model with the Flask Backend

The `app.py` file is the central component of the Flood Prediction System. It controls the interaction between the user interface and the machine learning model. The application is developed using the **Flask framework**, which provides a lightweight and efficient backend for handling HTTP requests and rendering web pages.

The trained **XGBoost model**, saved using Joblib, is loaded when the application starts. Whenever a user enters weather parameters such as annual rainfall, seasonal rainfall, cloud visibility, temperature, and humidity, the application processes these inputs, passes them to the trained model, and predicts whether the given conditions indicate a flood risk.

The prediction result is then displayed to the user through the web interface.

Objectives

The objectives of developing `app.py` are:

- Develop the backend using Flask.
- Load the trained machine learning model.
- Connect the frontend with the backend.
- Process user input.
- Generate real-time flood predictions.
- Display prediction results.

Main Functions of app.py

The main functionalities implemented in `app.py` include:

- Import required libraries.
- Initialize the Flask application.
- Load the trained XGBoost model.
- Receive user input.
- Predict flood occurrence.
- Display prediction results.

Step 1: Import Required Libraries

The required Python libraries are imported.

```
from flask import Flask, render_template, request
import numpy as np
import joblib
```

Step 2: Initialize Flask Application

Create the Flask application.

```
app = Flask(__name__)
```

Step 3: Load the Trained Model

The saved machine learning model is loaded during application startup.

```
model = joblib.load("flood_prediction_model.pkl")
```


Step 4: Create Home Route

The home page loads the input form.

```
@app.route('/')
def home():
    return render_template("index.html")
```

Step 5: Prediction Function

The prediction logic receives user inputs and generates flood prediction.

```
@app.route('/predict', methods=['POST'])
def predict():
    rainfall = float(request.form['rainfall'])
    seasonal = float(request.form['seasonal'])
    visibility = float(request.form['visibility'])
    temperature = float(request.form['temperature'])
    humidity = float(request.form['humidity'])
    prediction = model.predict([[rainfall,
                                seasonal,
                                visibility,
                                temperature,
                                humidity]])
    return render_template("result.html",
                           prediction=prediction)
```

Step 6: Run the Application

```
if __name__=="__main__":
    app.run(debug=True)
```

Activity 3.2: Implement Routes for Receiving User Inputs and Displaying Prediction Results

Once the backend application was developed, Flask routing was implemented to handle communication between the user interface and the machine learning model. Routes define how the application responds to different user requests. They receive weather parameters entered by users, send them to the prediction model, and return the prediction result.

The Flood Prediction System contains two primary routes:

- **Home Route (/)** – Displays the input form.
- **Prediction Route (/predict)** – Receives weather parameters, predicts flood risk, and displays the result.

These routes enable smooth interaction between the user and the machine learning model through the Flask framework.

Objectives

The objectives of route implementation are:

- Receive user inputs.
- Validate input values.
- Pass data to the machine learning model.
- Display prediction results.
- Improve user interaction.

Route 1: Home Route

The home route displays the application's main page.

```
@app.route("/")
def home():
    return render_template("index.html")
```

Route 2: Prediction Route

The prediction route receives the user inputs.

```
@app.route('/predict', methods=['POST'])
def predict():
    rainfall = request.form['rainfall']
    seasonal = request.form['seasonal']
    visibility = request.form['visibility']
    temperature = request.form['temperature']
    humidity = request.form['humidity']
    prediction = model.predict([[rainfall,
                                seasonal,
                                visibility,
                                temperature,
                                humidity]])
    return render_template("result.html",
                           prediction=prediction)
```

User Input Parameters

Parameter	Description
Annual Rainfall	Total rainfall received annually
Seasonal Rainfall	Rainfall during monsoon
Cloud Visibility	Cloud visibility percentage
Temperature	Atmospheric temperature
Humidity	Relative humidity

Table : Input Parameters

Milestone 4: Frontend Development

The objective of this milestone is to design and develop a user-friendly, responsive, and interactive web interface for the Flood Prediction System. The frontend allows users to enter weather parameters such as annual rainfall, seasonal rainfall, cloud visibility, temperature, and humidity through an intuitive form. The interface is built using **HTML**, **CSS**, and **JavaScript**, while Flask's **render_template()** function is used to dynamically display flood prediction results generated by the machine learning model.

Activity 4.1: Design and Develop the User Interface Using HTML, CSS, and JavaScript for Entering Weather Parameters

The frontend of the Flood Prediction System provides an easy-to-use interface through which users can enter weather-related information required for flood prediction. The interface is developed using **HTML** for page structure, **CSS** for styling and responsive design, and **JavaScript** for improving user interaction and input validation.

The homepage consists of a title, input form, prediction button, and navigation elements. The form allows users to enter important meteorological parameters that influence flood occurrence. The design follows a clean and responsive layout so that users can access the application from desktops, tablets, and mobile devices.

Objectives

The objectives of frontend development are:

- Design an attractive user interface.
- Create a responsive web page.
- Collect weather parameters from users.
- Improve user interaction.
- Validate input values before submission.

Frontend Technologies Used:

Technology	Purpose
HTML5	Structure of web pages
CSS3	Styling and responsive layout
JavaScript	Input validation and interaction
Flask	Backend communication

Table : Frontend Technologies

User Input Parameters

The interface collects the following weather parameters:

- Annual Rainfall
- Seasonal Rainfall
- Cloud Visibility
- Temperature
- Humidity

These parameters are entered through text fields or number input boxes.

Step 1: Create HTML Structure

The homepage (`index.html`) contains:

- Project title
- Weather parameter form
- Predict button
- Footer

Example:

```
<form action="/predict" method="POST">
<input type="number" name="rainfall">
<input type="number" name="seasonal">
<input type="number" name="visibility">
<input type="number" name="temperature">
<input type="number" name="humidity">
<button type="submit">
Predict Flood
</button>
</form>
```

Step 2: Apply CSS Styling

CSS is used to improve the appearance of the application.

Features include:

- Attractive background
- Responsive layout
- Styled buttons
- Rounded input boxes
- Hover effects
- Mobile-friendly design

Step 3: Add JavaScript Validation

JavaScript validates user inputs before submitting the form.

Example:

```
function validateForm(){  
  if(rainfall==""){  
    alert("Please enter rainfall");  
    return false;  
  }  
}
```

Activity 4.2: Create Dynamic Templates Using Flask's `render_template()` Function to Display Flood Prediction Results

After designing the user interface, dynamic templates were developed using Flask's `render_template()` function. These templates allow the backend to send prediction results directly to the frontend after processing user inputs.

When users submit weather parameters, Flask receives the data, loads the trained XGBoost model, predicts flood risk, and passes the prediction result to the HTML template. The webpage is then dynamically updated to display whether the area is **Flood Prone** or **Not Flood Prone**.

This dynamic interaction provides users with immediate prediction results without manually refreshing or modifying the webpage.

Objectives

The objectives of dynamic template development are:

- Display prediction results dynamically.
- Connect Flask backend with HTML templates.
- Improve user interaction.
- Provide real-time flood prediction.
- Enhance application usability.

Working of `render_template()`

The Flask `render_template()` function loads HTML pages and passes prediction results from the backend.

Example:

```
return render_template(  
  "result.html",  
  prediction=result)
```

Displaying Prediction Result

The HTML page displays the prediction.

Example:

```
<h2>  
Prediction Result  
</h2>  
<h3>  
{{ prediction }}  
</h3>
```

Milestone 5: Deployment

The objective of this milestone is to prepare the Flood Prediction System for deployment and make it accessible to users through a web server. The deployment process involves configuring the project environment, installing all required dependencies, organizing project files, and deploying the Flask web application either on **IBM Cloud** or a **local server**. Successful deployment enables disaster management authorities and users to access the application remotely and obtain real-time flood risk predictions.

Activity 5.1: Prepare the Application for Deployment by Configuring the Server Environment and Dependencies

Before deploying the Flood Prediction System, the application environment must be properly configured to ensure smooth execution. This includes installing the required Python libraries, organizing project files, creating dependency files, and verifying that the trained machine learning model and Flask application work correctly.

The deployment environment contains all the necessary resources required to execute the application, including the trained XGBoost model, Flask backend, HTML templates, CSS files, JavaScript files, and supporting Python libraries. Proper configuration ensures that the application runs consistently across different systems without compatibility issues.

Objectives

The objectives of deployment preparation are:

- Configure the deployment environment.
- Install all required dependencies.
- Organize project files.
- Verify application functionality.
- Prepare the project for cloud or local deployment.

Deployment Requirements

The following software components are required:

Software	Purpose
Python 3.10+	Programming Language
Flask	Web Framework
Pandas	Data Processing
NumPy	Numerical Computation
Scikit-learn	Machine Learning
XGBoost	Prediction Model
Joblib	Model Loading
HTML/CSS/JavaScript	Frontend Development

Table 5.1: Deployment Requirement

Project Directory Structure

The project files are organized as follows:

```
Flood Prediction Project
├── app.py
├── flood_prediction_model.pkl
├── requirements.txt
├── runtime.txt
├── Procfile
├── dataset/
│   └── flood.csv
├── templates/
│   ├── index.html
│   └── result.html
├── static/
│   ├── style.css
│   └── script.js
└── README.md
```

Step 1: Install Dependencies

Install all required Python libraries using:

```
pip install -r requirements.txt
```

Step 2: Generate Requirements File

Create a requirements file containing all installed libraries.

```
pip freeze > requirements.txt
```

Step 3: Verify Project Structure

Ensure all project files are available:

- app.py
- Model file (.pkl)
- Templates
- Static files
- Requirements file

Step 4: Test Application Locally

Run the Flask application.

```
python app.py
```

Verify that the application opens successfully in the browser.

Deployment Preparation Workflow

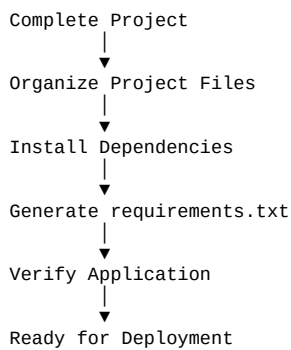


Figure 5.1: Deployment Preparation Workflow

Diagram 1: Deployment Environment Architecture

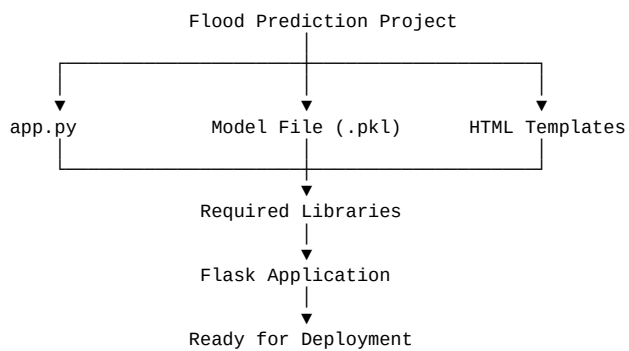


Figure 5.2: Deployment Environment Architecture

Diagram 2: Deployment Preparation Flowchart

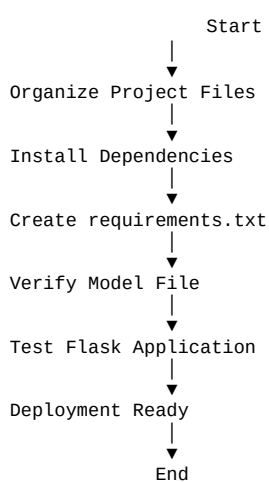


Figure 5.3: Deployment Preparation Flowchart

Activity 5.2: Deploy the Application on IBM Cloud or a Local Server to Enable Remote Access and Real-Time Predictions

After preparing the deployment environment, the Flood Prediction System was deployed to make it accessible for end users. The application can be hosted either on **IBM Cloud** or on a **local server** depending on project requirements. Deployment enables users to access the application through a web browser without installing any software locally.

The deployed Flask application loads the trained XGBoost model whenever a prediction request is received. Users enter weather parameters through the web interface, and the application instantly predicts whether the given conditions indicate a flood risk. This deployment supports real-time flood prediction and can assist disaster management authorities in making timely decisions.

Objectives

The objectives of deployment are:

- Host the Flood Prediction System online.
- Enable remote accessibility.
- Provide real-time flood predictions.
- Integrate the trained model with the deployed application.
- Ensure reliable and continuous application availability.

Deployment Process

Step 1: Prepare Deployment Files

Ensure the following files are available:

- app.py
- flood_prediction_model.pkl
- requirements.txt
- Procfile
- runtime.txt
- templates/
- static/

Step 2: Deploy Application

Deploy the project to IBM Cloud or host it on a local Flask server.

For local deployment:

```
python app.py
```

Step 3: Launch the Application

The application becomes accessible through a web browser.

Example:

```
http://127.0.0.1:5000
```

Step 4: Test Prediction

- Open the website.
- Enter weather parameters.

- Click **Predict**.
- View the flood prediction result.

Deployment Workflow

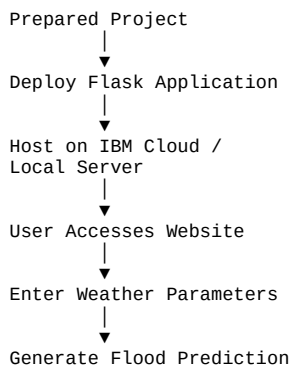


Figure 5.4: Deployment Workflow

Conclusion

The **Rising Waters** project was successfully developed to predict the likelihood of flood occurrence using historical weather data. The system utilized important meteorological parameters such as annual rainfall, seasonal rainfall, cloud visibility, temperature, and humidity to train multiple machine learning models, including **Decision Tree, Random Forest, K-Nearest Neighbours (KNN), and XGBoost**.

After evaluating the performance of all models, the **XGBoost algorithm** achieved the highest prediction accuracy of **96.55%** and was selected for deployment. The trained model was integrated into a **Flask-based web application**, providing users with a simple interface to enter weather parameters and receive instant flood risk predictions.

The developed system demonstrates that machine learning can significantly improve flood forecasting and support early warning systems. It can assist disaster management authorities in making timely decisions, reducing the impact of floods on human lives and property. In the future, the system can be enhanced by integrating real-time weather data, IoT sensors, GIS mapping, and cloud-based deployment to further improve its accuracy, scalability, and practical usefulness.